

MSML 641

Homework 2

Sampreeth Jangala

October 17, 2025

1 Methodology

1.1 Data Pre-processing

The foundational step for building a reliable language model is a robust data pre-processing pipeline. The goal is to transform the raw corpus text into a structured, numerical format that can be used to count N-gram frequencies accurately. For this assignment, the Penn Treebank dataset was processed using a series of key transformations applied to the training, validation, and test sets to ensure consistency.

- **Sentence Segmentation:** The dataset is structured such that each line represents a distinct sentence. This convention was used to segment the corpus, which is a critical prerequisite for N-gram modeling as probabilities should not be calculated across sentence boundaries.
- **Tokenization:** The corpus was pre-tokenized, with words and punctuation separated by spaces. The process involved splitting each sentence string into a list of its constituent tokens. This ensures that words like "director" and punctuation like "nov." are treated as separate entities.
- **Adding Sentence Boundary Markers:** A crucial step for N-gram models is to explicitly model the beginning and end of sentences. To achieve this, two special tokens were added to every sentence. A start-of-sentence token, `<s>`, was prepended, and an end-of-sentence token, `</s>`, was appended. This allows the model to calculate probabilities like $P(\text{the}|\text{<s>})$, the likelihood of a sentence starting with "the," and $P(\text{</s>}|\text{group})$, the likelihood of a sentence ending after the word "group."
- **Handling Unknown Words:** The training data contains a pre-defined `<unk>` token. This token is used to represent all out-of-vocabulary (OOV) words that appear in the validation or test sets but were not observed during training. By mapping all OOV words to this single token, the model can assign a non-zero probability to sentences containing novel words, preventing the model from failing entirely.

The transformation is illustrated below. A raw sentence is converted into a list of tokens ready for N-gram extraction.

Code Listing 1: Example of a sentence before and after pre-processing.

```
# Raw sentence from the dataset:
# "mr. <unk> is chairman of <unk> n.v. the dutch publishing group"

# After processing, it becomes a list of tokens:
[ '<s>', 'mr.', '<unk>', 'is', 'chairman', 'of', '<unk>',
  'n.v.', 'the', 'dutch', 'publishing', 'group', '</s>' ]
```

1.2 Corpus Statistics and N-gram Frequencies

After pre-processing, an analysis of the training corpus was performed to understand its statistical properties. The corpus consists of 42,068 sentences. The frequencies of unigrams, bigrams, and trigrams were calculated, revealing key characteristics of the data that directly influence model performance.

Unigram Statistics The vocabulary size, defined by the number of unique unigrams, is 10,001. As shown in Table 1, the most frequent tokens include common English stop words like ('the'), the special tokens added during pre-processing (<s>, </s>), and the unknown token <unk>. The high frequency of <unk> (45,020 occurrences) immediately highlights the prevalence of rare words in the corpus, foreshadowing the data sparsity challenges for higher-order models.

Table 1: Top 5 Most Frequent Unigrams in the Training Corpus.

Unigram	Frequency
('the')	50,770
('<unk>')	45,020
('<s>')	42,068
('</s>')	42,068
('N')	32,481

Higher-Order N-gram Statistics As the order of N increases, the number of unique grams grows exponentially, while the frequency of any single N-gram decreases. The training data contains 264,990 unique bigrams and 586,558 unique trigrams. The most common higher-order N-grams, shown in Table 2, reflect the financial nature of the Wall Street Journal corpus, with patterns like ('\$ ', 'N') and ('\$ ', 'N', 'million') being highly frequent.

Table 2: Top 5 Most Frequent Bigrams and Trigrams in the Training Corpus.

Bigram	Frequency	Trigram	Frequency
('\$', 'N')	7,475	('\$', 'N', 'million')	3,606
('N', 'N')	7,183	('\$', 'N', 'billion')	1,486
('<s>', 'the')	7,057	('N', 'to', 'N')	1,151
('of', 'the')	5,290	('N', 'N', 'to')	1,141
('in', 'the')	4,503	('to', '\$ ', 'N')	1,061

This explosive growth in unique N-grams is the root cause of the data sparsity problem. While the model can be confident about the probability of a frequent sequence like ('\$ ', 'N', 'million'), it is statistically guaranteed that the vast majority of valid English trigrams will not appear in the training data at all. This motivates the need for smoothing techniques, which are discussed in the following sections.

1.2.1 Maximum Likelihood Estimation (MLE)

The baseline approach for N-gram modeling is Maximum Likelihood Estimation. The core assumption of MLE is that the probability of a sequence can be estimated directly from the frequencies observed in the training corpus. The conditional probability of a word w_i given its preceding context is calculated as the count of the full N-gram divided by the count of the N-1 context:

$$P_{MLE}(w_i|w_{i-n+1} \dots w_{i-1}) = \frac{C(w_{i-n+1} \dots w_i)}{C(w_{i-n+1} \dots w_{i-1})}$$

This method is intuitive and works well for N-grams that are frequent in the training data. For instance, to calculate the probability of the word <unk> following the context <s>

`pierre`, the model correctly computes a probability of 0.5 based on the observed counts (Listing 2).

Code Listing 2: Example MLE calculation for a seen trigram.

— Test 1: A trigram present **in** the training data —

Trigram: ('`<s>`', '`pierre`', '`<unk>`')

Numerator $C(' <s> ', ' pierre ', ' <unk> ')$: 1

Denominator $C(' <s> ', ' pierre ')$: 2

Model Probability: $1 / 2 = 0.5$

Result: SUCCESS

However, the critical weakness of MLE becomes apparent when the model encounters a sequence not present in the training corpus. In this case, the N-gram count is zero, and the model assigns the sequence a probability of zero, as demonstrated in Listing 3.

Code Listing 3: Demonstration of the zero-probability problem for an unseen trigram.

— Test 2: An unseen trigram —

Trigram: ('`this`', '`is`', '`unseen`')

Numerator $C(' this ', ' is ', ' unseen ')$: 0

Denominator $C(' this ', ' is ')$: [Could be > 0 , but numerator **is** 0]

Model Probability: 0.0

Result: SUCCESS (Correctly identifies unseen n-gram)

While technically correct according to the formula, assigning a probability of zero is highly problematic for a language model. It implies that unseen sequences are impossible, which is an incorrect assumption for any real-world text. This "zero-probability problem" makes the model brittle and leads to an infinite perplexity when evaluated on new data, necessitating the use of smoothing techniques.

2 Experimental Results and Analysis

2.1 Impact of N-gram Order on MLE Models

To establish a baseline, Maximum Likelihood Estimation (MLE) models were trained for N-gram orders from N=1 (unigram) to N=4 (quadgram). Each model was then evaluated on the held-out test set to calculate its perplexity. The results, summarized in Table 3, highlight a critical challenge in language modeling.

Table 3: Perplexity of unsmoothed MLE models on the test set. The models for $N > 1$ fail due to unseen N-grams in the test data.

Model (N-gram Order)	Perplexity (PP)
1-gram (Unigram)	576.98
2-gram (Bigram)	∞
3-gram (Trigram)	∞
4-gram (Quadgram)	∞

The trend is stark and immediate: while the unigram model produces a finite (though high) perplexity, all higher-order models ($N > 1$) fail, resulting in an infinite perplexity. This failure is a direct consequence of the **data sparsity** problem inherent in natural language.

As the N-gram order increases, the model's context becomes more specific. For example, a 4-gram model makes predictions based on the three preceding words. While this stronger **Markov Assumption** should theoretically lead to more accurate predictions, it also exponentially increases the number of possible N-grams. The training corpus, despite its size, represents only a tiny fraction of all possible valid English sequences.

Consequently, the test set inevitably contains N-grams that were never observed during training. The log output shows this explicitly, noting that the bigram ('no', 'it') was the first unseen sequence to cause the 2-gram model to fail. According to the MLE formula, the probability of an unseen N-gram is zero. Since the perplexity calculation involves the product of probabilities (or the sum of their logs), a single zero-probability event makes the entire result undefined, reported here as infinity. This result clearly demonstrates that while MLE is a foundational concept, it is insufficient on its own for building robust language models, and that **smoothing** techniques are not just an improvement, but a necessity.

2.2 Smoothing Technique 1: Add-1 (Laplace) Smoothing

To address the failure of the MLE models, the first and most straightforward smoothing technique, Add-1 (or Laplace) smoothing, was implemented for the trigram ($N=3$) model. This method solves the zero-probability problem by adding one to the count of every N-gram, both seen and unseen. The probability formula is adjusted as follows:

$$P_{\text{Add-1}}(w_i | w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i) + 1}{C(w_{i-2}, w_{i-1}) + V}$$

Here, V represents the total vocabulary size, which was determined to be 10,001 from the training data. By adding 1 to the numerator, it guarantees that even an unseen N-gram (with a count of 0) will have a non-zero probability. The denominator is adjusted by adding the entire vocabulary size to account for all the "phantom" counts added in the numerator.

Results and Discussion The Add-1 smoothed trigram model was evaluated on the test set, achieving a perplexity of 3748.66.

The key success of this method is that it produces a finite perplexity score. The model no longer fails when it encounters an N-gram that was not in the training data. For example, the previously "impossible" trigram ('this', 'is', 'unseen') is now assigned a small but non-zero probability of approximately 9.8×10^{-5} .

However, the quality of the model, as measured by perplexity, is extremely poor. The score of ≈ 3749 is significantly worse than even the baseline unigram MLE model (PP ≈ 577). This demonstrates the major drawback of Add-1 smoothing: it is too blunt. It reassigns a very large amount of probability mass from the N-grams that were actually observed to the massive set of N-grams that were never seen. While this solves the mathematical problem, it creates a model that fits the training data poorly and does not generalize well, motivating the need for more sophisticated smoothing techniques.

2.3 Smoothing Technique 2: Linear Interpolation

A more sophisticated approach than Add-1 smoothing is Linear Interpolation. This technique improves upon simple MLE by combining evidence from multiple N-gram orders. Instead of relying solely on the often-sparse trigram counts, it creates a weighted average of the trigram, bigram, and unigram probabilities. This allows the model to "fall back" on more reliable, lower-order statistics when the higher-order context is rare. The probability is calculated as:

$$P_{\text{interp}}(w_i | \dots) = \lambda_3 P_{MLE}(w_i | w_{i-2}, w_{i-1}) + \lambda_2 P_{MLE}(w_i | w_{i-1}) + \lambda_1 P_{MLE}(w_i)$$

The coefficients $(\lambda_1, \lambda_2, \lambda_3)$ are hyperparameters that control the weight given to each N-gram model, and they are constrained to sum to 1.

Hyperparameter Tuning The optimal values for the λ weights are data-dependent and must be determined experimentally. To do this without tainting the final test results, the **validation set** was used. Several combinations of λ weights were tested, with the goal of finding the combination that minimized perplexity on the validation data. The results of this tuning process are shown in Table 4.

Table 4: Tuning of λ weights on the Validation Set.

Lambda Combination $(\lambda_1, \lambda_2, \lambda_3)$	Validation Perplexity
[0.1, 0.3, 0.6]	259.16
[0.33, 0.33, 0.34]	206.87
[0.6, 0.3, 0.1]	223.38
[0.1, 0.8, 0.1]	230.53

The tuning process identified the nearly-even weighting of [0.33, 0.33, 0.34] as the optimal combination, achieving the lowest perplexity on the validation set.

Final Results and Discussion Using the optimal weights found during tuning, the final Linear Interpolation model was evaluated on the held-out test set. The model achieved a perplexity of **192.14**.

This result is a dramatic improvement over the Add-1 smoothing model ($PP \approx 3749$). By blending the predictive power of different N-gram orders, the model is far more robust. When a specific trigram is rare or unseen, the model can still rely on the more frequently occurring bigram and unigram components to make a reasonable prediction. This validates that leveraging lower-order statistics is a highly effective strategy for combating data sparsity.

2.4 Smoothing Technique 3: Stupid Backoff

The final strategy implemented was **Stupid Backoff**. Unlike Linear Interpolation, which blends probabilities from different N-gram orders, Stupid Backoff uses a simpler, hierarchical approach. It trusts the highest-order N-gram model (trigram) exclusively if it has a non-zero probability for a given sequence. If the trigram is unseen, it "backs off" to the bigram model, multiplying its probability by a fixed decay factor, α . If the bigram is also unseen, it backs off again to the unigram model, applying the decay factor once more.

The score, S , is calculated as follows:

$$S(w_i|w_{i-2}w_{i-1}) = \begin{cases} P_{MLE}(w_i|w_{i-2}w_{i-1}) & \text{if } C(w_{i-2}w_{i-1}w_i) > 0 \\ \alpha \cdot P_{MLE}(w_i|w_{i-1}) & \text{else if } C(w_{i-1}w_i) > 0 \\ \alpha^2 \cdot P_{MLE}(w_i) & \text{otherwise} \end{cases}$$

It is important to note that this method does not produce a true probability distribution (the scores do not sum to 1), but it is known to be a computationally efficient and effective method for ranking sequences, which is sufficient for calculating perplexity.

Hyperparameter Tuning Similar to the λ weights in interpolation, the backoff factor α is a hyperparameter that was tuned on the validation set. Several values for α were evaluated to find the one that minimized perplexity, with the results shown in Table 5.

Table 5: Tuning of the backoff factor α on the Validation Set.

Alpha (α)	Value	Validation Perplexity
0.1		610.53
0.4		202.12
0.7		129.36
1.0		97.34

Interestingly, the optimal value was found to be $\alpha = 1.0$, indicating that for this dataset, no penalty for backing off to a lower-order model yielded the best performance on the validation set.

Final Results and Discussion Using the optimal alpha of 1.0, the Stupid Backoff model was evaluated on the test set. It achieved a perplexity of **91.38**.

This result is a significant improvement over both Add-1 smoothing and Linear Interpolation, establishing Stupid Backoff as the best-performing model in this Homework. The success of this simpler, non-probabilistic method highlights that for the task of ranking next-word predictions, the strict constraints of a true probability distribution may not be necessary. The strategy of trusting the most specific context available and only backing off when necessary is highly effective and computationally efficient.

2.5 Qualitative Analysis of Generated Text

Quantitative metrics like perplexity are essential for comparing models, but a qualitative analysis of the model’s output provides insight into its capabilities and limitations. Using the best-performing model; the Stupid Backoff model with $\alpha = 1.0$ five sample sentences were generated.

The generated sentences are as follows:

- regular prime-time wednesday and a share recently as head from moscow rose N million \$ in again banks <s> in addition good company had similar the american flag has created a most real estate unit is sure <unk> would depend an affluent people and mitsubishi ’s investment adviser
- <unk> protein
- had net income toward the conservative mr. richard parent owe in stock-index futures exchange earned \$ N million

- the guys closing they have <unk> group profit earliest <s> if he on the week level since the largest cities yields
- the coaches

Discussion The generated text exhibits a phenomenon common to N-gram models: **local fluency but global incoherence**. The model is proficient at generating short, syntactically plausible sequences of words. For example, phrases like “had net income,” “investment adviser,” and “stock-index futures” are coherent and contextually appropriate for the financial news domain. This is because these common trigrams (or bigrams) appeared frequently in the training data, and the model assigns them a high probability.

However, the model lacks any true understanding of semantics or long-term context. Its memory is limited to the preceding two words (for a trigram model). As a result, the sentences quickly become nonsensical streams of loosely-connected phrases. The generation process is purely statistical: given a context of N-1 words, the model calculates a probability distribution over the entire vocabulary for the next word and samples from it. The window then slides forward one token, and the process repeats. This explains why the sentences can feel “stitched together” and why artifacts like the start-of-sentence token <s> can occasionally appear mid-sentence if that sequence had a non-zero probability. The variable sentence lengths are also a natural outcome of this process, as the end-of-sentence token </s> is simply sampled like any other word.

3 Conclusion

This project successfully implemented and evaluated a series of N-gram language models, providing a clear demonstration of the challenges and solutions inherent in statistical language modeling. The initial experiments with unsmoothed Maximum Likelihood Estimation models starkly illustrated the critical issue of data sparsity; while the unigram model produced a baseline perplexity, all higher-order models ($N > 1$) failed to produce a finite score on the test set, confirming that the zero-probability problem makes unsmoothed models practically unusable.

The necessity of smoothing was clearly established, with subsequent experiments revealing a wide performance gap between different techniques. Add-1 (Laplace) smoothing, while mathematically solving the zero-probability issue, performed exceptionally poorly ($PP \approx 3749$), highlighting its overly simplistic approach. In contrast, more sophisticated methods that leverage lower-order N-gram information proved far more effective. Linear Interpolation significantly improved performance ($PP \approx 192.14$), and ultimately, the Stupid Backoff model, tuned with an alpha of 1.0 on the validation set, proved to be the most effective, achieving a final test perplexity of **91.38**.

Beyond the quantitative perplexity scores, the qualitative analysis of the generated text highlighted the inherent limitations of the N-gram approach. The best model could produce locally fluent phrases but lacked any long-term coherence, a direct result of the fixed, short-term memory defined by the Markov assumption. In conclusion, this work demonstrated the full lifecycle of building an N-gram model: from establishing a flawed baseline to implementing, tuning, and rigorously evaluating advanced backoff techniques to create a functional and well-performing model. The results reaffirm that the success of a classical language model hinges not just on the order of N, but critically on the sophistication of its smoothing strategy.